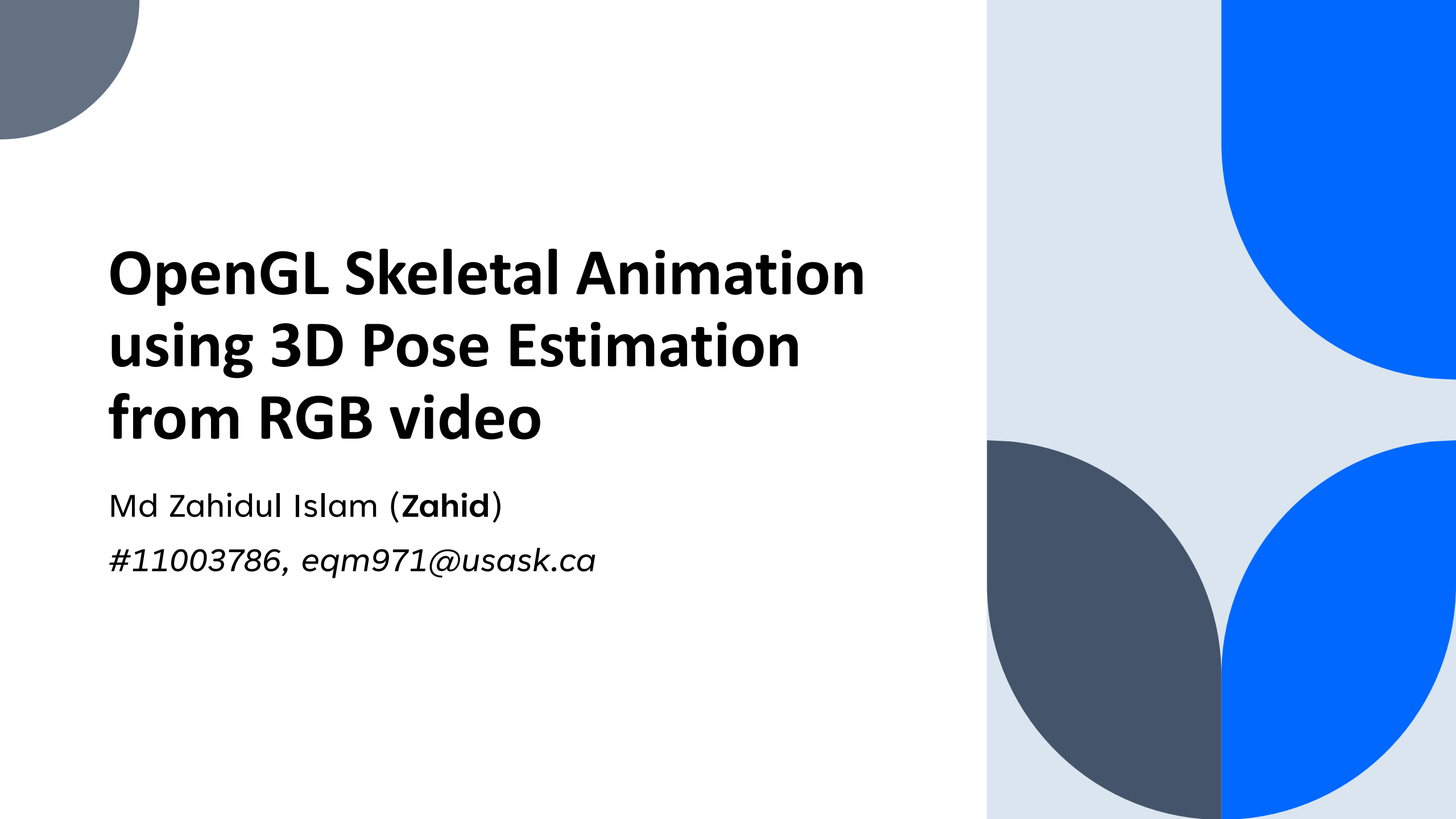




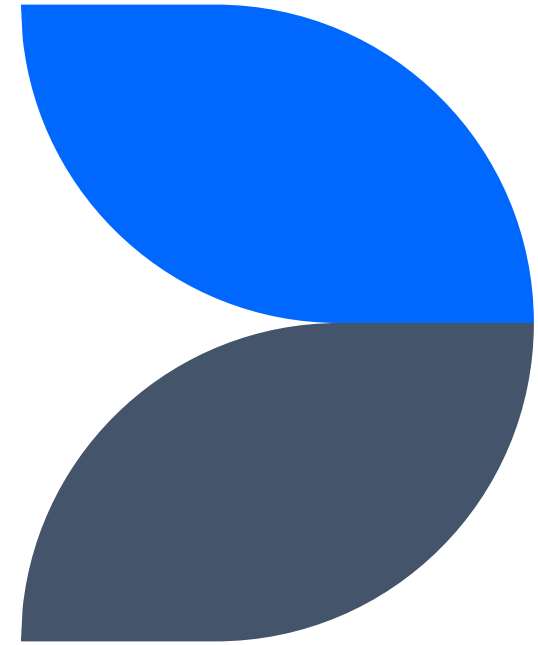
OpenGL Skeletal Animation using 3D Pose Estimation from RGB video

Md Zahidul Islam (**Zahid**)

#11003786, eqm971@usask.ca

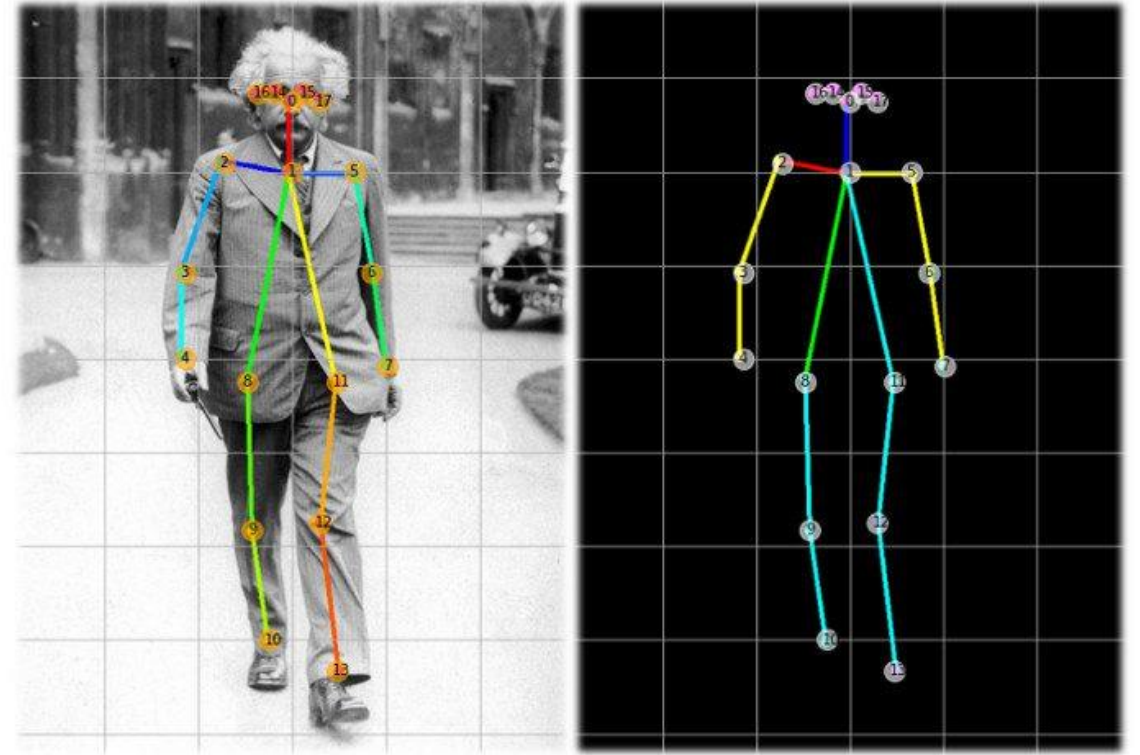
Introduction

- Pose Estimation
- Skeletal Animation



Pose Estimation

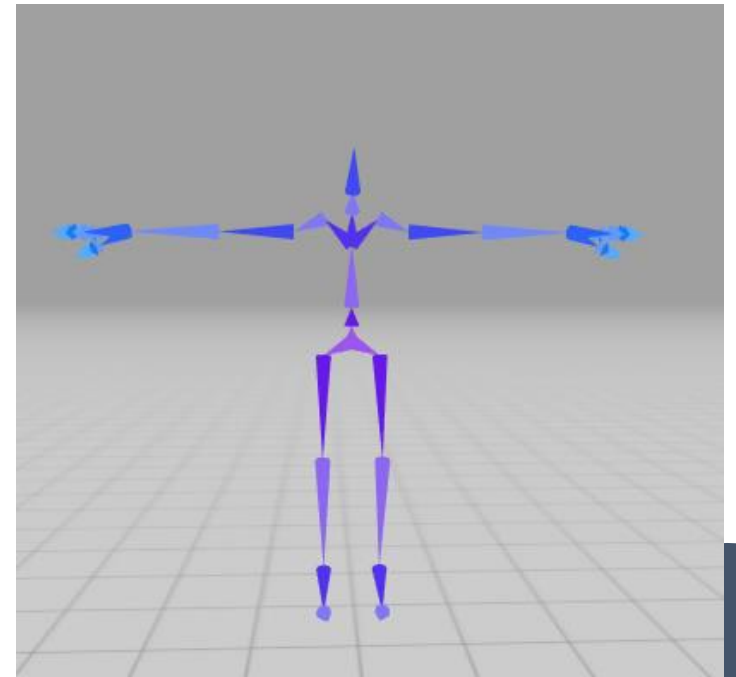
- Input : RGB Image or Video
- Output: 2D or 3D Joint Co-ordinates
- State-of-the-art Deep Learning methods:
 - **MotionBERT**, OpenPose, HRNet



Source: <https://mobidev.biz/blog/human-pose-estimation-technology-guide>

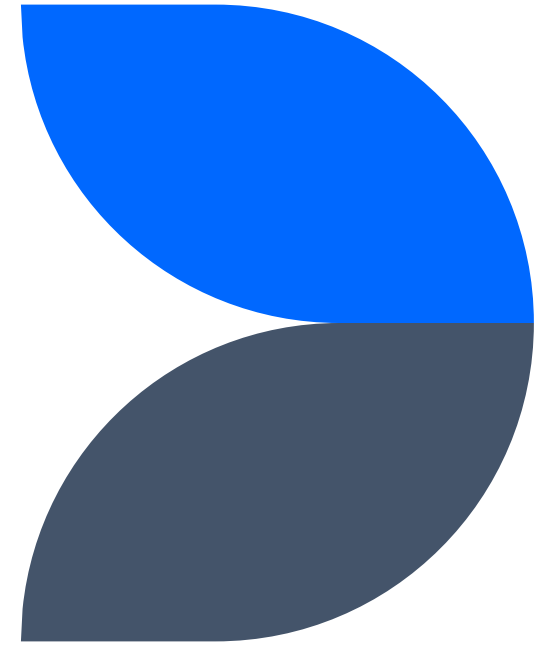
Skeletal Animation

- Move/transform different parts of the model with respect to each other.
- **Bones:**
 - Controls the motion/transformation of a neighboring vertices.
 - Is not rendered
 - Parent-children hierarchy.
 - Characterized by a Position and Orientation.
 - Defined with respect to the parent.
 - Root bone (usually Hip-center) doesn't have a parent.



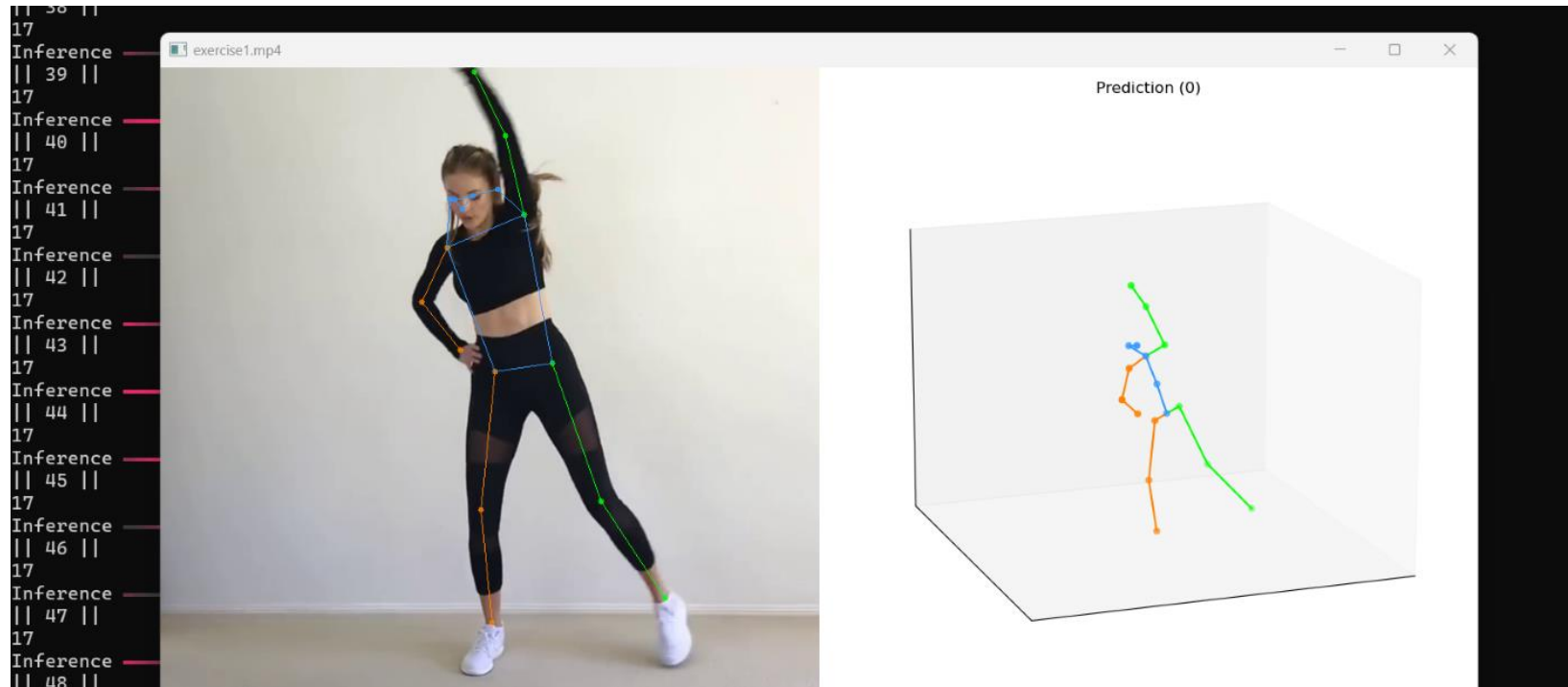
Methodology

- 3D Pose Estimation
- Joint Angle Calculation
- Skeletal Animation



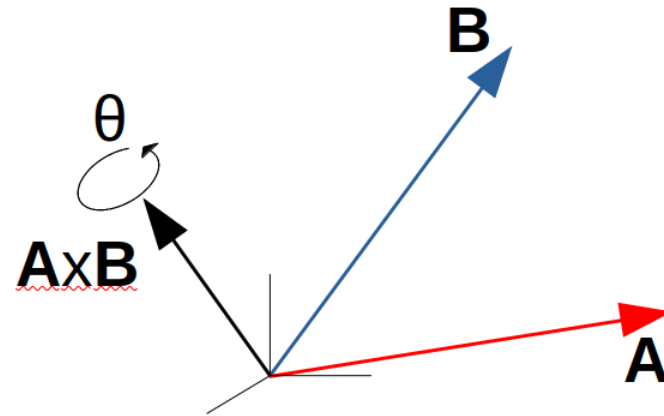
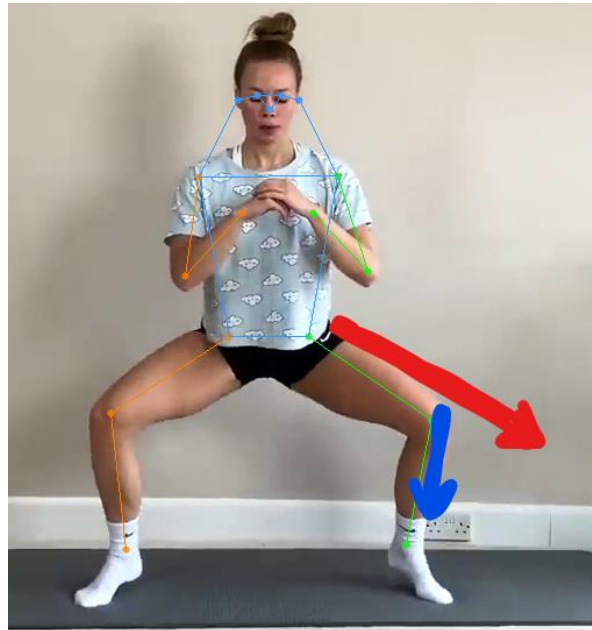
3D Pose Estimation

- Used **MMPose** and **PyTorch** python library.
- For each frame in video, got 3D co-ordinates 17 body joints.
- 0:'Root', 1:'RightHip', 2:'RightKnee', 3:'RightAnkle', 4:'LeftHip', 5:'LeftKnee', 6:'LeftAnkle', 7:'Torso', 8:'Neck', 9:'Nose', 10:'Head', 11:'LeftShoulder', 12:'LeftElbow', 13:'LeftWrist', 14:'RightShoulder', 15:'RightElbow', 16:'RightWrist'



Joint-Angle Calculation

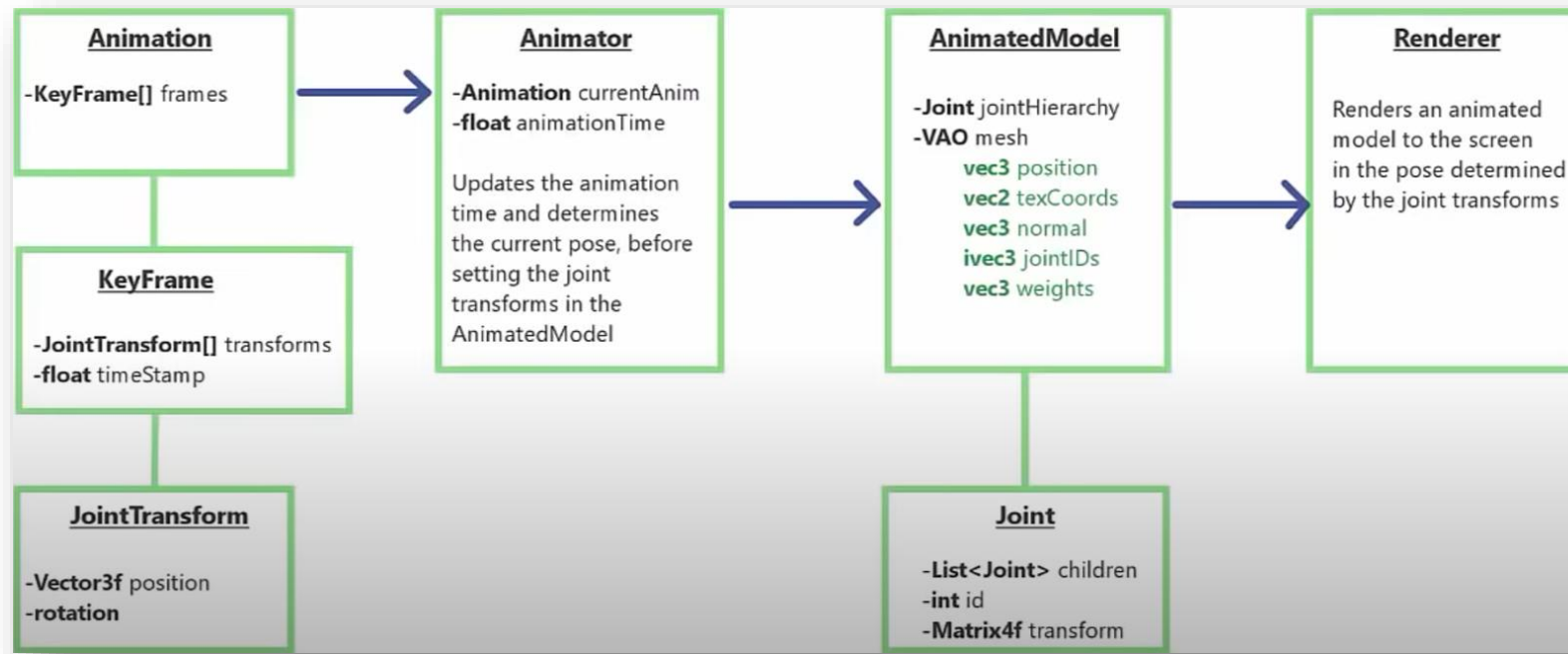
- OpenGL expects the relative orientation of each bone with its parent, not joint-coords.
- Easier to apply to a model with different body size.
- Convert Euler-Angles (**vec3**) to Quaternions (**vec4**) and send to OpenGL.
 - Using Euler-angles give erroneous results due to Gimbal Lock.



Find a 3D-Rotation that transforms the Red vector in the Blue one. That will be the local orientation of the Blue bone with respect to its parent, Red bone.

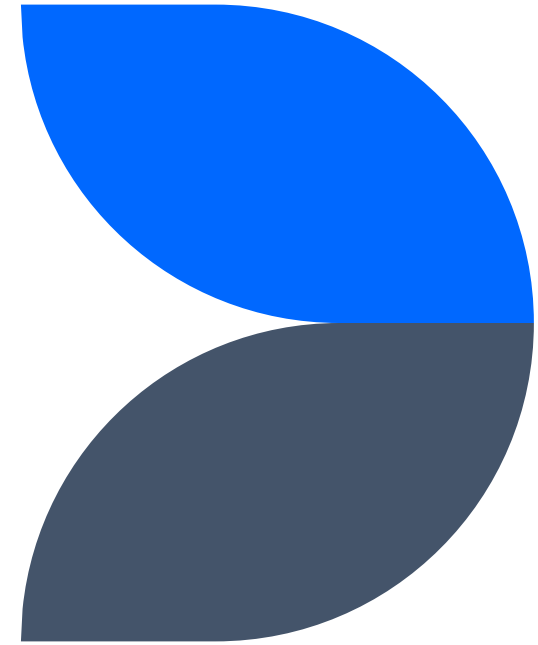
Skeletal Animation

- For each bone,
 - Get all bones along its parent to the root following the hierarchy.
 - Multiply their local transform matrices to get a final global transformation for the bone. Use this transform to move neighboring vertices.

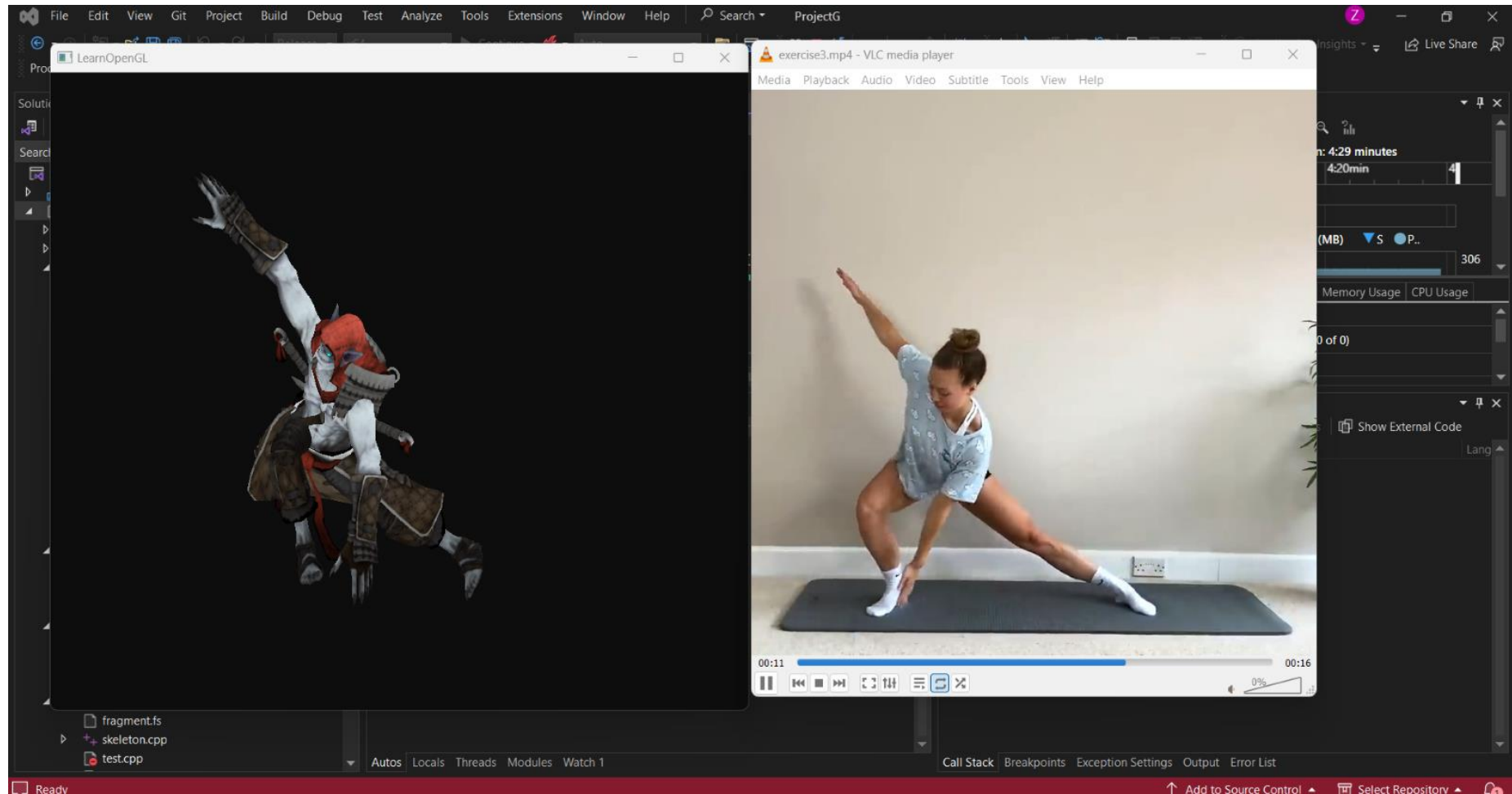


Class definitions and their relationships

Results & Demo



Results - Good

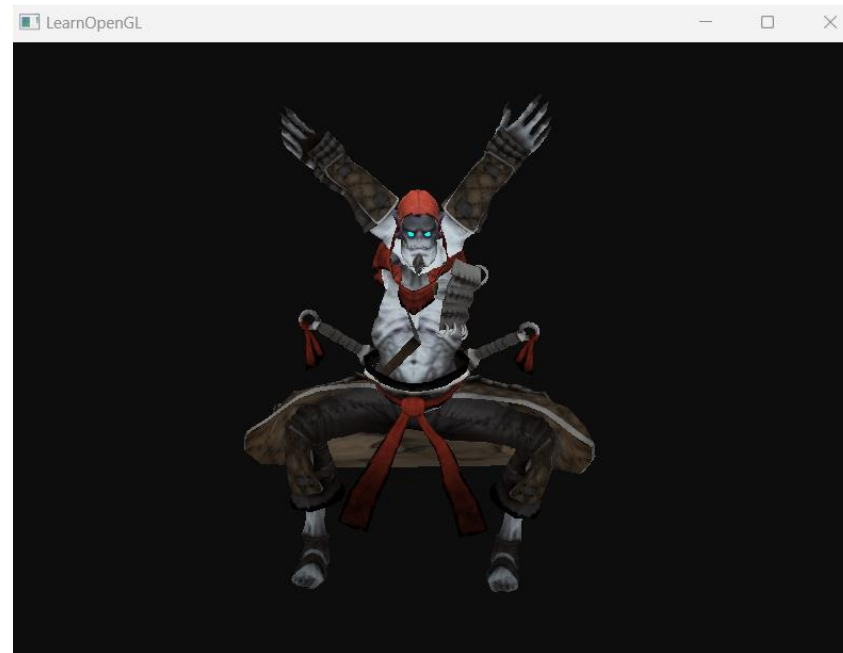
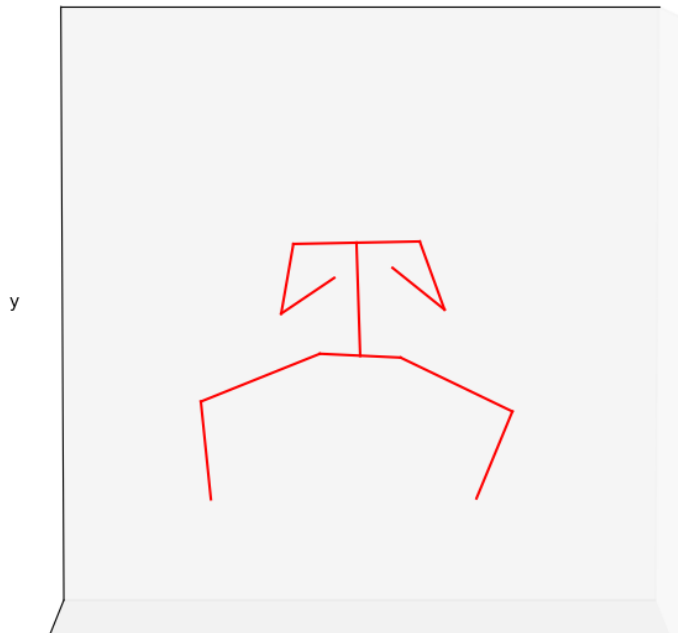


Results - Bad

- For certain poses, produces wonky results.

Probable causes:

- Mismatch between body proportion of the human and the skeletal model
- Need to limit the range of orientation for each bone.
- Erroneous results or Limited number of joints in 3D pose detector.



Thank You

References

[1] MotionBERT: A Unified Perspective on Learning Human Motion Representations, Zhu, Wentao and Ma, Xiaoxuan and Liu, Zhaoyang and Liu, Libin and Wu, Wayne and Wang, Yizhou, Proceedings of the IEEE/CVF International Conference on Computer Vision, 2023

[2] OpenMMLab Pose Estimation Toolbox and Benchmark, MMPose Contributors, <https://github.com/open-mmlab/mmpose>, 2023

- Following documentations, tutorials, and starter codes from the following online resources were utilized in this project.

<https://www.youtube.com/watch?v=f3Cr8Yx3GGA&t=605s>

<https://www.youtube.com/watch?v=GZQkwx10p-8&t=5181s>

https://mmpose.readthedocs.io/en/latest/user_guides/inference.html

<https://motionbert.github.io/>

<https://learnopengl.com/Guest-Articles/2020/Skeletal-Animation>

https://temugeb.github.io/python/motion_capture/2021/09/16/joint_rotations.html

- Following youtube videos were utilized as input data-

<https://www.youtube.com/watch?v=9g4z0gFlfpA&t=480s>

<https://www.youtube.com/watch?v=WTUruNwUMFI&t=319s>